APPLIED DATA SCIENCE CAPSTONE

Predicting Falcon 9 first-stage landing success

An end-to-end analysis of SpaceX launch data

IBM Skills Network course dataset | 90 labeled launches

IBM Applied Data Science Capstone

2026

Executive summary

The course sample supports a reproducible comparison of four classifiers. Landing outcomes vary across sites and mission profiles, while the small labeled sample limits how strongly any pattern can generalize.

90
labeled launches

60 / 30
successful / failed landings

83.3%
best holdout accuracy

88.0%
best holdout F1

Decision-useful takeaway

A decision tree ranked first in cross-validation and achieved 15 correct classifications out of 18 holdout launches. Treat this as a course-scale result, not an operational forecast.

The analysis combines API collection, historical scraping, SQL and visual EDA, Folium, Dash, and supervised classification.

Problem and project question

SpaceX advertises Falcon 9 launches at a substantially lower cost when it can recover and reuse the first stage. The analysis asks which mission and vehicle attributes are associated with a successful landing in the course dataset.

TARGET

Class = 1

First-stage landing succeeded

PREDICTORS

80 encoded features

Payload, orbit, launch site, booster version, and mission indicators

SCOPE

Descriptive analysis and a held-out classification exercise. Observed differences do not establish causal effects.

API collection: retrieve and normalize launch records

- Request archived SpaceX launch JSON supplied by IBM Skills Network; retain raw response for reproducibility.
- Parse each launch and its first-stage core; extract date, flight, rocket, payload, launchpad and landing fields.
- Normalize nested JSON to a 107-row, 18-column CSV; preserve raw payload and failure JSON for traceability.

FLOW: IBM course archive → parse JSON + nested core → normalize selected fields → CSV (107 × 18)

Web scraping: extract dated launch-history tables

- Use a fixed Wikipedia revision (987426997, 6 Nov 2020) so the source is stable and auditable.
- Read page tables, identify flight-number and date columns, remove citation marks, then parse dates as UTC.
- Keep numbered launches from 4 Jun 2010 through 5 Nov 2020; drop invalid dates and duplicate flight numbers.
- Result: 97 historical launch rows, saved with the source-revision label in the output CSV.

SCRAPE FLOW

Pinned revision → pandas.read_html → identify + parse → date filter + deduplicate → CSV

Wrangling aligns features, labels and launch context

- Validate the supplied 90 × 80 one-hot feature matrix against the matching IBM labeled course table by FlightNumber.
- Decode Orbit, LaunchSite, LandingPad and Serial; confirm every decoded category matches the source row.
- Use the source table Class as target (60 successful, 30 failed); fill missing landing-pad labels as Unknown.
- Keep API history, scrape history and the fixed 90-row model sample separate because their schemas and coverage differ.

QUALITY CHECKS

Row count + flight order · one-hot sums · decoded category equality · target class counts

EDA and interactive analytics methods

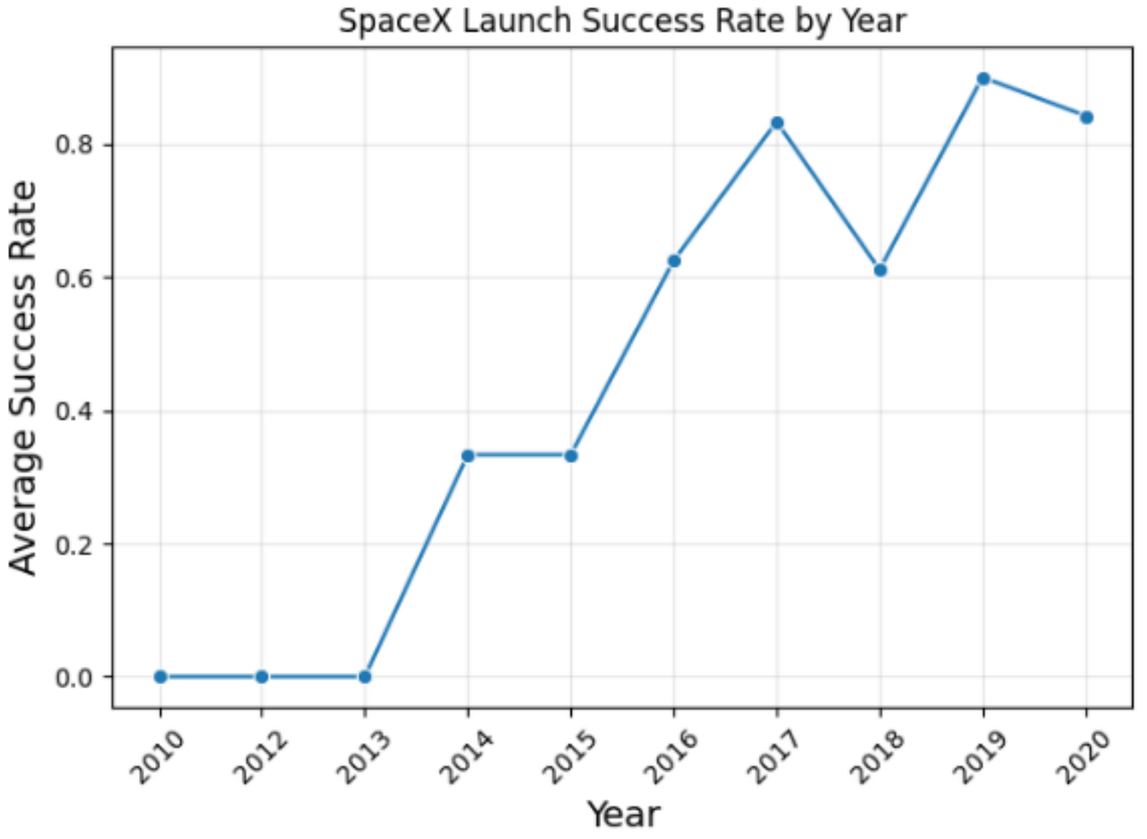
Pandas + Matplotlib	Compare outcome patterns by flight number, year, payload, launch site, and orbit.
SQLite	Aggregate counts, success rates, and mean payload by site, orbit, and outcome.
Folium	Place launch sites on an interactive geographic map with outcome summaries.
Plotly Dash	Filter the labeled sample by launch site and orbit; inspect rates and payload distributions.

Classification design

Data split	72 train / 18 test	Stratified, random_state = 2
Model selection	5-fold stratified CV	Training set only
Algorithms	Logistic, SVM, KNN, tree	Grid search per model
Scaling	Pipeline for logistic, SVM, KNN	Tree uses raw encoded features
Selection metric	Mean CV accuracy	Final metrics reported on holdout

The holdout remains untouched during model selection. This separation reduces optimistic test estimates.

Landing outcomes improved across the observed years



Observed pattern

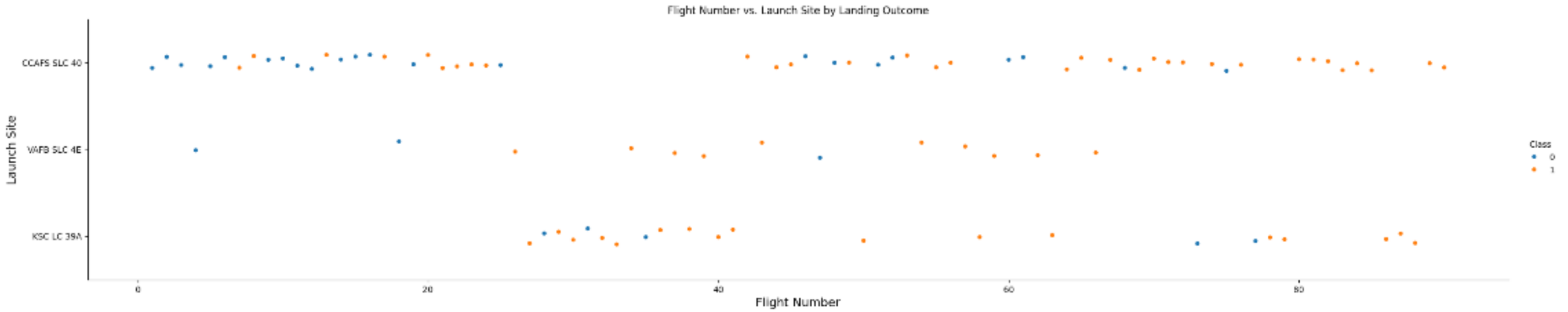
The course sample moves from no recorded successes in its early years to rates above 80% in 2017 and 2019–2020.

Interpretation

The year trend also reflects fleet maturity, mission mix, and changes in operations. It does not isolate the effect of time alone.

Sample: 90 labeled launches | yearly denominator varies

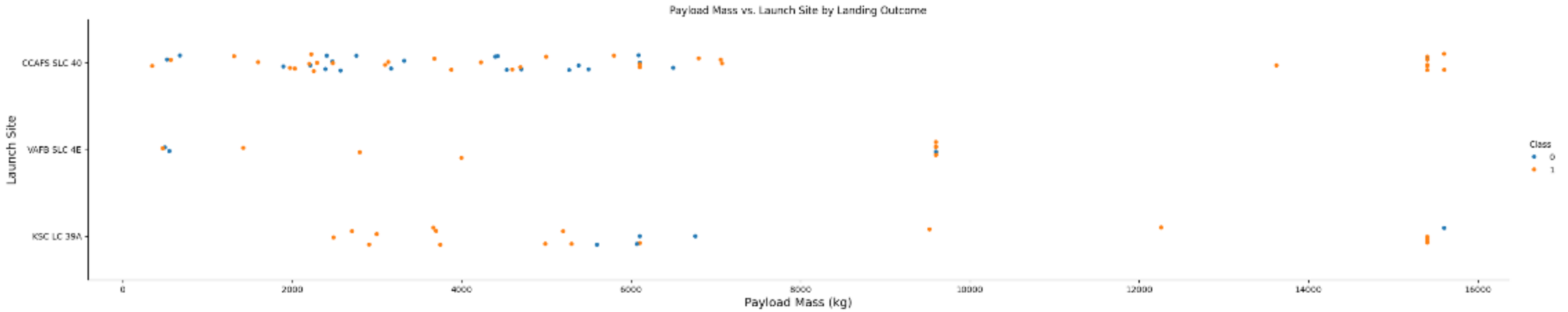
Launch-site outcomes appear in different flight-number ranges



Each point is one launch. Failures and successes occupy different parts of the flight sequence at each site, but site, year, booster version, and mission type change together.

Source: 90 labeled launches

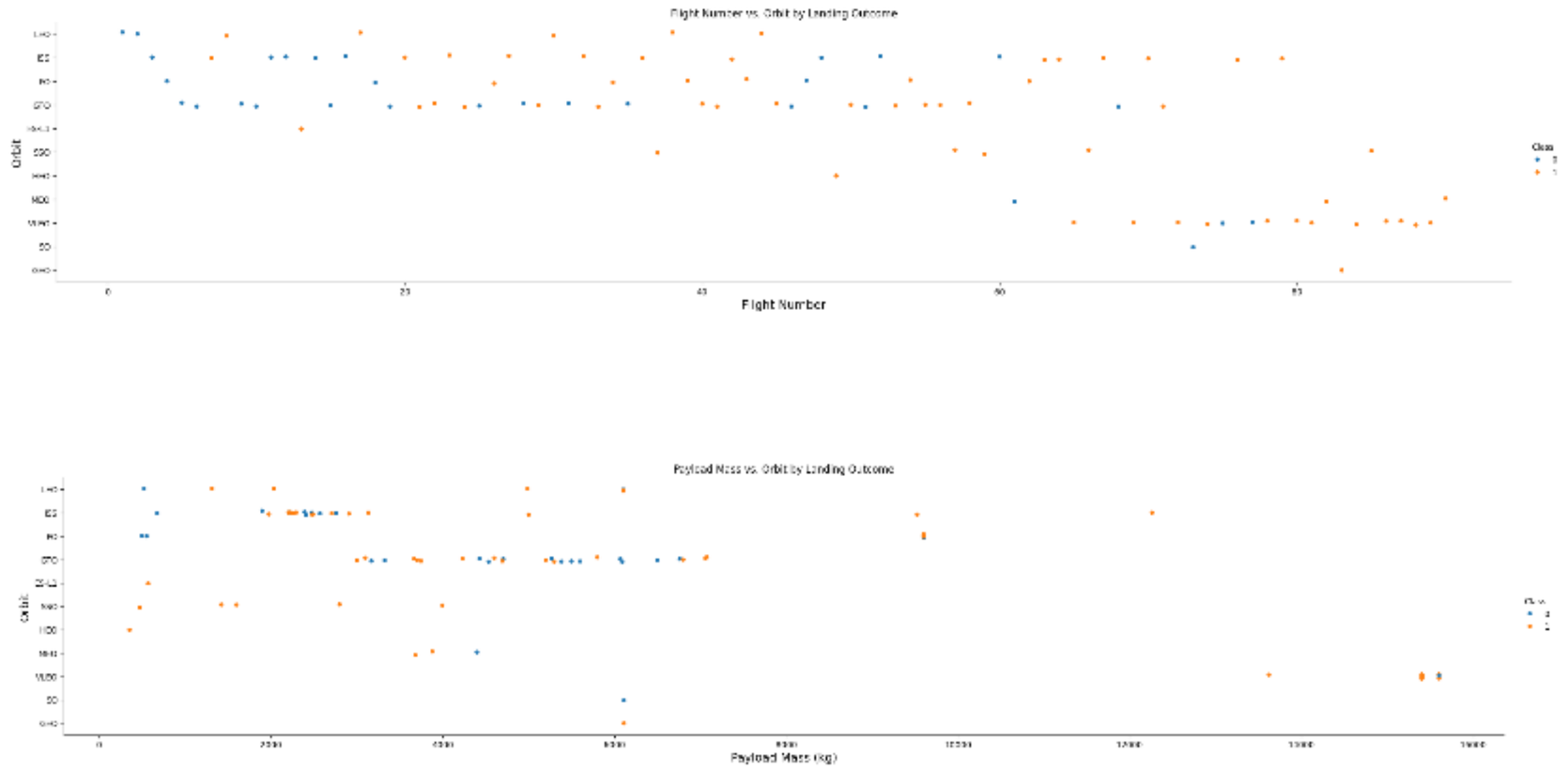
Payload mass alone does not separate outcomes by site



Both outcomes occur across a wide payload range. Several high-mass launches succeeded, so payload weight alone does not separate outcomes.

The mean payload is higher among successes, but this comparison does not adjust for orbit or year.

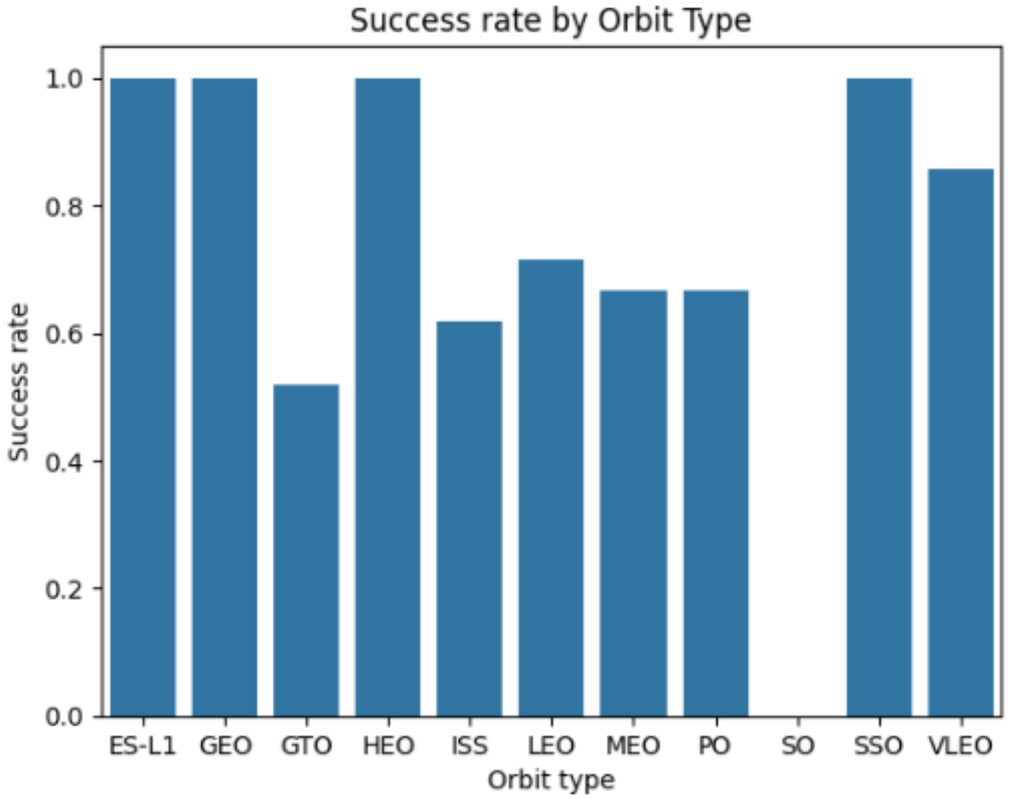
Orbit and payload patterns depend on sample size



Some orbit categories have very few launches. A 100% observed rate with one launch does not imply a stable high-success profile.

The larger groups, such as GTO and ISS, show both outcomes and support more informative comparisons.

Bar charts compare observed success rates across categories



Why this chart

The bars make orbit-level outcome differences easy to compare at a glance.

Read with counts

Rare orbits can show 100% success with only one launch. SQL tables report denominators alongside the rates.

Additional EDA visuals: flight-versus-site and payload-versus-site scatter plots; annual success-rate trend.

SQL analysis makes group sizes explicit

SQLite queries were run against the same 90-row labeled course sample. Each result reports group size beside the rate so rare categories remain visible.

- Launch-site rates: GROUP BY LaunchSite, with COUNT(*) and AVG(Class).
- Orbit rates: GROUP BY Orbit, ordered by observed success rate.
- Payload comparison: AVG(PayloadMass) grouped by Class.
- Rates are descriptive; denominators and mission mix differ across groups.

Reproducible query file: outputs/sql/capstone_queries.sql

KSC LC 39A and VAFB SLC 4E lead CCAFS SLC 40 in this sample

Launch site	Launches	Successful	Observed rate
KSC LC 39A	22	17	77.3%
VAFB SLC 4E	13	10	76.9%
CCAFS SLC 40	55	33	60.0%

The 17.3-point gap between the highest and lowest site rates is an unadjusted sample difference. Site, era, booster, orbit, and payload overlap.

Orbit rates need their denominators

Orbit	Launches	Success rate
GTO	27	51.9%
ISS	21	61.9%
VLEO	14	85.7%
LEO	7	71.4%
PO	9	66.7%
MEO	3	66.7%
SSO	5	100.0%
GEO	1	100.0%
HEO	1	100.0%
ES-L1	1	100.0%
SO	1	0.0%

Payload mean by observed outcome

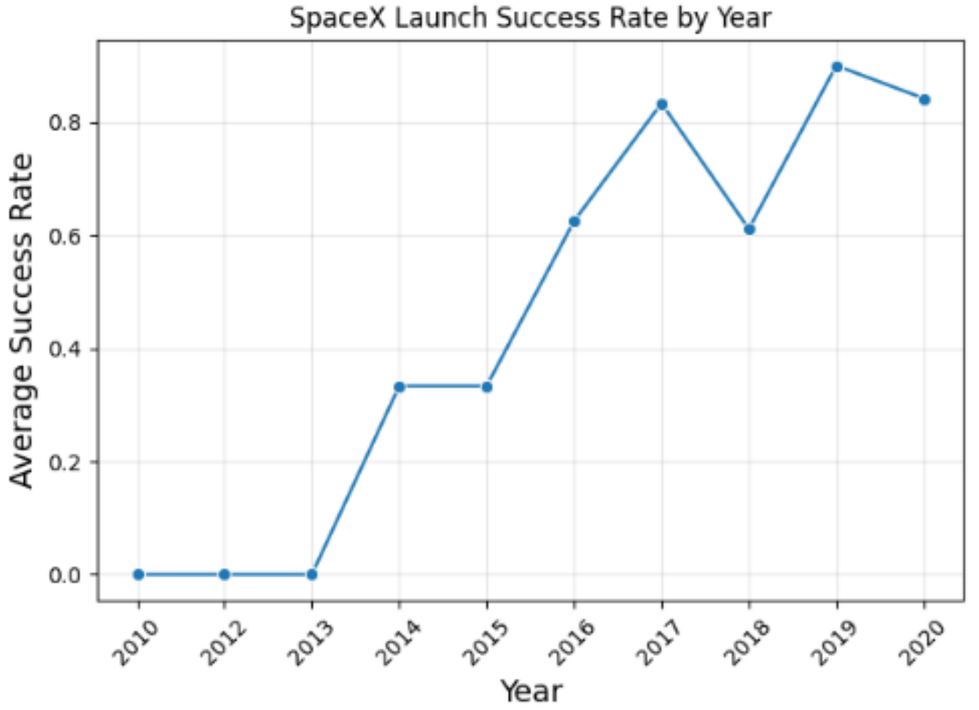
Failure 4,784.6 kg

Success 6,765.2 kg

The difference is not causal evidence. Orbit and payload distributions vary, and successful launches include heavy payloads.

Temporal SQL analysis tracks volume and landing success by year

The query extracts the UTC year from each launch date, then reports launch count, successful launches and the observed success rate.



The time pattern shows a marked increase after the early Falcon 9 years.

SQL result:
outputs/sql/year_success.csv

Interpret with the yearly launch counts: small denominators make annual rates volatile.

Folium maps the three sampled launch sites

The project generates an interactive HTML map with one marker per launch site. Marker popups report launch counts and observed landing rates.

Site	Sample launches	Observed success
CCAFS SLC 40	55	60.0%
KSC LC 39A	22	77.3%
VAFB SLC 4E	13	76.9%

Interactive artifact

outputs/launch_sites_folium.html

Open the HTML file in a browser to zoom and inspect site markers.

Proximity analysis: KSC LC 39A and CCAFS SLC 40 are about 5.6 km apart; VAFB SLC 4E is about 3,720 km from the nearest Florida site.

Each popup reports site, launch records, successful launches, rate and nearest sampled site distance.

Dash exposes the sample through interactive filters

The Plotly Dash app contains launch-site and orbit filters with two linked views.

Filter	Launch-site and orbit dropdowns restrict the displayed sample.
View 1	Pie chart shows the share of successful and failed launches in the filtered sample.
View 2	Scatter plot compares payload mass with binary landing outcome; hover reveals flight, site and orbit.
Run	python src/dash_app.py, then open http://127.0.0.1:8050

The app layout, route, and filtered callback were exercised with the Flask test client.

PREDICTIVE RESULTS

Decision tree ranked first in training cross-validation

Model	CV accuracy	Test accuracy	Precision	Recall	F1
Logistic regression	83.7%	77.8%	78.6%	91.7%	84.6%
SVM	82.3%	77.8%	78.6%	91.7%	84.6%
KNN	69.4%	66.7%	66.7%	100.0%	80.0%
Decision tree	87.7%	83.3%	84.6%	91.7%	88.0%

CV selects the tree. Its holdout result is 15 / 18 correct, with 12 / 12 successes detected and 4 / 6 failures detected.

PREDICTIVE RESULTS

The holdout contains only six failed landings

Actual \ Predicted	Failure (0)	Success (1)
Failure (0)	4	2
Success (1)	1	11

Model

Decision tree
Gini, max depth 5

Recall: 91.7%
Precision: 84.6%
F1: 88.0%

One additional error among six failures changes failure-class metrics sharply. Treat the model as a learning exercise, not a launch decision system.

Conclusion: mission context matters alongside payload

What the sample supports

Landing success becomes more common in later years in this dataset. Site and orbit rates differ, but their group sizes and mission mix also differ.

Model result

A tuned decision tree led the four-model comparison on five-fold CV accuracy and the 18-row holdout.

Innovative insight

The apparent payload pattern reverses a simple “lighter is easier” story: average payload was higher among successes, while success still varied by orbit and launch site. Mission context and vehicle maturity deserve joint analysis.

Next analysis: validate on a larger, time-aware sample and measure uncertainty within site-orbit groups.

Reproducibility and submission files

- Seven notebooks document collection, wrangling, EDA, SQL, Folium, Dash, and modeling.
- Python scripts, SQL queries, source datasets, output tables, visualizations, and the interactive map are organized in the project folder.
- The presentation PDF is the file to upload to the Mark question.

Repository URL

https://github.com/joaoltrevisan/spacex-falcon9-capstone

The repository contains the completed notebooks, source code, data, SQL outputs, and presentation.

Sources: IBM Skills Network course datasets; SpaceX API archive; Wikipedia revision 987426997 (2020-11-06).